

AIM:

To create and access the database using triggers.

TRIGGER:

A database trigger is a stored procedure that is fired when an insert, update or delete statement is issued against the associated table. Database triggers can be used for the following purposes.

- To generate data automatically.
- To enforce integrity constraints.
- To customize security authorizations.
- To maintain replicate tables.

Syntax for creating Triggers:

Create or Replace trigger<trigger name>

[before/after][insert/update/delete] on <table_name>

[for each statement/for each row]

[when <condition>]

TO CREATE A TABLE

```
SQL> create table staff_details (staff_id number(3), full_name varchar2(10),  
joining_date date, pay_per_month number(6));
```

Table created.

TO DISPLAY THE CONTENTS OF THE TABLE

```
SQL> select * from staff_details;
```

STAFF_ID	FULL_NAME	JOINING_D	PAY_PER_MONTH
101	akshay	01-JAN-23	18000
102	nisha	15-MAR-23	22000
103	yokarajan	10-NOV-22	30000
104	dev	01-JUN-24	45000
105	surya	20-AUG-23	12000

TO CREATE TRIGGER AND UPDATE THE SALARY VALUE:

SQL> create or replace trigger salary_update

```
2   before insert on staff_details
3   for each row
4   begin
5       if :new.pay_per_month <= 30000 then
6           :new.pay_per_month := 25000;
7       end if;
8   end;
9   /
```

Trigger created.

TO INSERT VALUES INTO THE TABLE REVISED:

SQL> insert into staff_details values(&staff_id, '&full_name',
to_date('&joining_date','yyyy-mm-dd'), &pay_per_month);

Enter value for staff_id: 106

Enter value for full_name: Ram

Enter value for joining_date: 2024-05-15

Enter value for pay_per_month: 35000

old 1: insert into staff_details values(&staff_id, '&full_name',
to_date('&joining_date','yyyy-mm-dd'), &pay_per_month)

new 1: insert into staff_details values(106, 'Ram', to_date('2024-05-15','yyyy-mm-dd'), 35000)

1 row created.

TO DISPLAY THE TABLE REVISED AFTER UPDATING:

SQL> select * from staff_details;

STAFF_ID	FULL_NAME	JOINING_D	PAY_PER_MONTH
101	akshay	01-JAN-23	18000
102	nisha	15-MAR-23	22000
103	yokarajan	10-NOV-22	30000
104	dev	01-JUN-24	45000
105	surya	20-AUG-23	12000
106	Ram	15-MAY-24	35000

6 rows selected.

TO DROP THE TRIGGER CREATED:

```
SQL> drop trigger salary_update;
```

Trigger dropped.

TO CREATE A TABLE NEWBACKUP:

```
SQL> create table newbackup(staff_id number(3), full_name varchar2(10), joining_date date, pay_per_month number(6));
```

Table created.

TO CREATE A TABLE OLDBACKUP:

```
SQL> create table oldbackup(staff_id number(3), full_name varchar2(10), joining_date date, pay_per_month number(6));
```

Table created.

TO CREATE ANOTHER TRIGGER:

```
SQL> create or replace trigger t2
```

```
2  after insert or update on staff_details
```

```
3  for each row
```

```
4  begin
```

```
5      if inserting then
```

```
6          insert into newbackup (staff_id, full_name, joining_date, pay_per_month)
```

```
7          values (:new.staff_id, :new.full_name, :new.joining_date,
:new.pay_per_month);
```

```
8          dbms_output.put_line('inserted: ' || :new.staff_id || ', ' ||
:new.full_name);
```

```
9      elsif updating then
```

```
10         insert into oldbackup (staff_id, full_name, joining_date, pay_per_month)
```

```
11         values (:old.staff_id, :old.full_name, :old.joining_date,
:old.pay_per_month);
```

```
12         dbms_output.put_line('updated: ' || :old.staff_id || ', ' || :old.full_name);
```

```
13     end if;
```

```
14 end;
```

```
15 /
```

Trigger created.

TO INSERT VALUES INTO THE TABLE REVISED:

```
SQL> insert into staff_details values (107, 'Kamal', to_date('2024-06-10', 'yyyy-mm-dd'), 35000);
```

1 row created.

TO DISPLAY THE TABLE TRIG AFTER UPDATING:

SQL> select * from staff_details;

STAFF_ID	FULL_NAME	JOINING_D	PAY_PER_MONTH
101	akshay	01-JAN-23	18000
102	nisha	15-MAR-23	22000
103	yokarajan	10-NOV-22	30000
104	dev	01-JUN-24	45000
105	surya	20-AUG-23	12000
106	Ram	15-MAY-24	35000
107	Kamal	10-JUN-24	35000

7 rows selected.

TO DISPLAY THE TABLE NEWBACKUP:

STAFF_ID	FULL_NAME	JOINING_D	PAY_PER_MONTH
107	Kamal	10-JUN-24	35000

TO UPDATE THE FIELD SALARY IN THE TABLE TRIG:

SQL> update staff_details set pay_per_month = 30000 where pay_per_month <= 20000;

2 rows updated.

TO DISPLAY THE TABLE TRIG:

SQL> select * from staff_details;

STAFF_ID	FULL_NAME	JOINING_D	PAY_PER_MONTH
101	akshay	01-JAN-23	30000
102	nisha	15-MAR-23	22000
103	yokarajan	10-NOV-22	30000
104	dev	01-JUN-24	45000
105	surya	20-AUG-23	30000
106	Ram	15-MAY-24	35000
107	Kamal	10-JUN-24	35000

7 rows selected.

TO DISPLAY THE TABLE OLDBACKUP:

SQL> select * from oldbackup;

STAFF_ID	FULL_NAME	JOINING_D	PAY_PER_MONTH
----------	-----------	-----------	---------------

101	akshay	01-JAN-23	18000
105	surya	20-AUG-23	12000

PROGRAMS:

```
SQL> create table employee_info (emp_id number(3), emp_name varchar2(10),
monthly_salary number(6));
```

Table created.

```
SQL> insert into employee_info values (101, 'akshay', 18000);
```

1 row created.

```
SQL> insert into employee_info values (102, 'nisha', 22000);
```

1 row created.

```
SQL> select * from employee_info;
```

EMP_ID	EMP_NAME	MONTHLY_SALARY
--------	----------	----------------

101	akshay	18000
102	nisha	22000
103	yokarajan	25000
104	dev	27000
105	surya	16000

```
SQL> create or replace trigger salary_check_trigger
```

```
2   before insert on employee_info
```

```
3   for each row
```

```
4   declare
```

```
5       min_salary constant number := 5000;
```

```
6   begin
```

```
7       if :new.monthly_salary < min_salary then
```

```
8           raise_application_error(-20001, 'salary cannot be less than ' ||
min_salary);
```

```
9       end if;
```

```
10  end;
```

11 /

Trigger created.

```
SQL> insert into employee_info values(106, 'ram', 4500);
```

```
insert into employee_info values(106, 'ram', 4500)
```

*

ERROR at line 1:

ORA-20001: salary cannot be less than 5000

ORA-06512: at "SYSTEM.SALARY_CHECK_TRIGGER", line 5

ORA-04088: error during execution of trigger 'SYSTEM.SALARY_CHECK_TRIGGER'

2. CREATE A ROW LEVEL AFTER UPDATE TRIGGER TO INSERT

THE NEW VALUES INTO ANOTHER TABLE ALSO SHOW THE

UPDATED VALUE IN ANOTHER TABLE:

CREATE TABLE emp_audit:

```
SQL> create table emp_audit (audit_id number(3), emp_id number(3), emp_name  
varchar(10), monthly_salary number(6));
```

Table created.

CREATE SEQUENCE :

```
SQL> create sequence emp_audit_seq start with 1 increment by 1;
```

Sequence created.

CREATE TRIGGER – AFTER UPDATE:

```
SQL> create or replace trigger after_salary_update
```

```
2  after update on employee_info
```

```
3  for each row
```

```
4  begin
```

```
5  insert into emp_audit(audit_id, emp_id, emp_name, monthly_salary)
```

```
6  values (emp_audit_seq.nextval, :new.emp_id, :new.emp_name, :new.monthly_salary);
```

```
7  end;
```

```
8  /
```

Trigger created.

UPDATE ONE VALUE IN sums TABLE:

SQL> update employee_info set monthly_salary = 8000 where emp_id = 102;

1 row updated.

emp_audit TABLE:

SQL> select * from emp_audit;

AUDIT_ID EMP_ID EMP_NAME MONTHLY_SALARY

1 102 nisha 8000

CONTENTS	MARKS ALLOTTED	MARKS OBTAINED
Aim, Algorithm, SQL, PL/SQL	30	
Execution and Result	20	
Viva	10	
Total	60	

RESULT:

Thus various types of trigger are created for the database